



ENTERPRISE APPLICATION DEVELOPMENT

EAPD7111

FORMATIVE 1 PART 1

Ramathabathe Windy Phasha
ST10501179

EXECUTIVE SUMMARY

This report presents the complete technical strategy for the Tech Move GLMS project. It covers three key areas: (1) selecting and applying an enterprise architecture framework, (2) three software design patterns for C# code bases, and (3) non-functional requirements including availability, compatibility, and scalability. This report recommends TOGAF as the architecture, selects Observer, Factory Method, and Decorator as the design patterns, and sets an availability goal of 99.9% with the ability to process 50,000 new contracts overnight.

1: ARCHITECTURAL FRAMEWORK STRATEGY

Before creating a large software system, you need a plan. An enterprise architecture framework is like a blueprint to help you create this plan. The Tech Move GLMS project considered two popular platforms: Zachman and TOGAF. (Standard, 2018)

Zachman's framework is like a checklist. This will help you ask the right questions (What? How?). shape? or? when? Why?) from different perspectives (boss, designer, builder). But Zachman does not tell you in what order things should be done. It is great for organizing information, but it is not great for guiding you through a project step by step. TOGAF is different. It provides a clear step-by-step process called ADM (Architecture Development Method). This process takes you from a big idea to a final working system. Each step has a specific purpose and produces a specific result.

Why we chose **TOGAF for Tech Move**: Tech Move currently uses manual methods to track deliveries and contracts. This causes errors and delays. The company needs a clear and actionable plan to move from manual to automated management. The board would like to see progress at a clear stage. TOGAF offers just that. It is more like a cooking recipe than just a list of ingredients. Therefore, TOGAF is the right choice for this project. (Zachman J. , 1987)

Phase A: Big picture (architectural vision)

The first phase of TOGAF answers one question: what are we trying to build and why?

First, I will identify everyone who is interested in the new system:

The **board** wants to know that their money is being spent wisely. Operations managers use this system daily. They want it to be simple and reliable. Drivers need a simple mobile app to receive delivery instructions.

Customers always want to know where their products are. Latest issue: Tech Move uses paper forms, spreadsheets, and phones to track shipments. This leads to incorrect

information, lost documents, and customer dissatisfaction. Goal: Create a cloud-based GLMS that automates everything from signing contracts to confirming deliveries. The system must also seamlessly connect to Tech Move's existing ERP (accounting) and CRM (client) systems so that data is transferred automatically. What you get from this step: Two documents. Formal approval is given to start the project. Some people explain their vision in simple terms so that everyone understands their goals.

Phase B: Business Structure (Business Architecture)

In this step, we will look at how Tech Move works now and how it will work after the new system is installed.

Current workflow (step by step):

- Conclude a contract with a client.
- Plan your delivery route.
- Assign driver.
- Track your package.
- Confirm delivery.
- Send invoice.

Every step now requires manual data entry. This is where the mistake happens. What motivated this project?

Three basic needs:

1. **Growth:** Tech Move wants to attract many new customers without disrupting existing processes.
2. **Accuracy:** Manual entry will result in tracking and billing errors. The system needs to fix this.
3. **Visibility:** Today's customers expect to be able to see where their products are in real time. Without it, Tech Move will lose business.

Who does what? New systems must support a variety of roles.

- **Dispatcher:** Decide which pilot receives which baggage.
- **Warehouse Manager:** Monitors inventory accuracy.
- **Driver:** Update delivery status from the road.
- **Accountant:** Automatically generate invoices upon delivery.

Understanding these roles ensures that the system provides the right tools for each person's job.

Phase C – Information and Software (Information Systems Architecture)

This phase has two parts: data (what information) and application (what software).

Data architecture – What information will be stored in the system?

The system follows these basic elements:

- **Contract:** Customer name, type of service (standard, expedited, or fragile items), cost, and deadline.
- **Shipping:** destination, origin, current location, and status (receiving, in transit, delivering).
- **Vehicle:** Truck number, size, and last trip time.
- **Driver:** name, phone number, license details, current employment.
- **Client:** billing address, contact person and payment terms.
- **Warehouse:** location of the warehouse and its current availability.

There are also rules. For example: "A package cannot be shipped until a driver is assigned." Another rule: "The contract must be approved before supplies can be made under the contract. These rules prevent mistakes.

Application architecture. What software modules do I need?

GLMS has several modules.

- **Route optimization engine:** Find the fastest and cheapest shipping routes.
- **Contract Management Module:** Manage different contract types and approval rules.
- **Notification service:** Send automatic updates to customers (text message or email) and drivers.
- **API Gateway:** Allows other systems (such as accounting) to interact with the GLMS.
- **Reporting Module:** Displays dashboards and performance reports for managers.

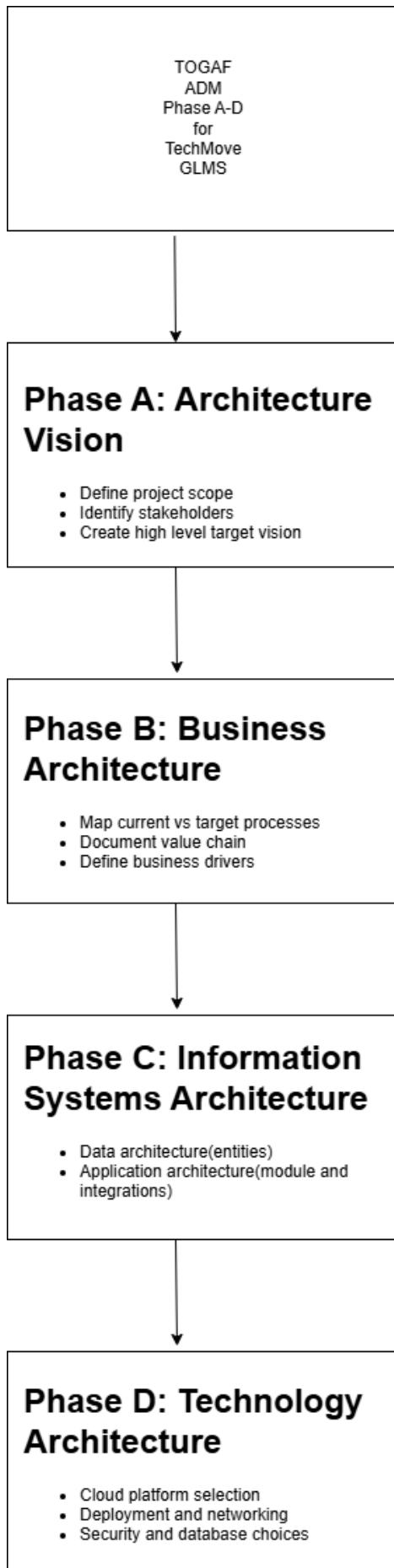
Phase D – Real Technology (Technology Architecture)

At this point, the logical design turns into an actual technical choice.

- **Cloud platform:** Microsoft Azure. For what? It is logistics friendly, stores data in multiple locations for security, and complies with South African data laws.
- **How it works:** The system uses Docker containers (that is, small portable software packages) managed by Kubernetes. This means that if one part is busy (e.g. tracking during the holiday season), that part can automatically increase without affecting other parts.
- **API (method of communication between systems):** two types. REST for simple daily operations. GraphQL when customers need complex data in a single query.
- **Databases:**

- Azure SQL for important transactional data (contracts, shipments, invoices). This database ensures that no data is lost or corrupted.
- Redis as a fast memory cache of current delivery status. This requires the system to react quickly.
- Security:
 - **Role-based access control:** Drivers cannot view financial statements. Accountants cannot change driver assignments. Each person sees only what he needs.
 - **Encryption:** All network traffic is blocked using TLS.
 - **Authentication:** External systems must connect using OAuth 2.0 tokens (secure digital keys).

This technology plan gives the development team clear standards to follow. It also gives the board confidence that the system will be secure, able to grow, and easy to maintain.



(Zachman T. v., 2024)

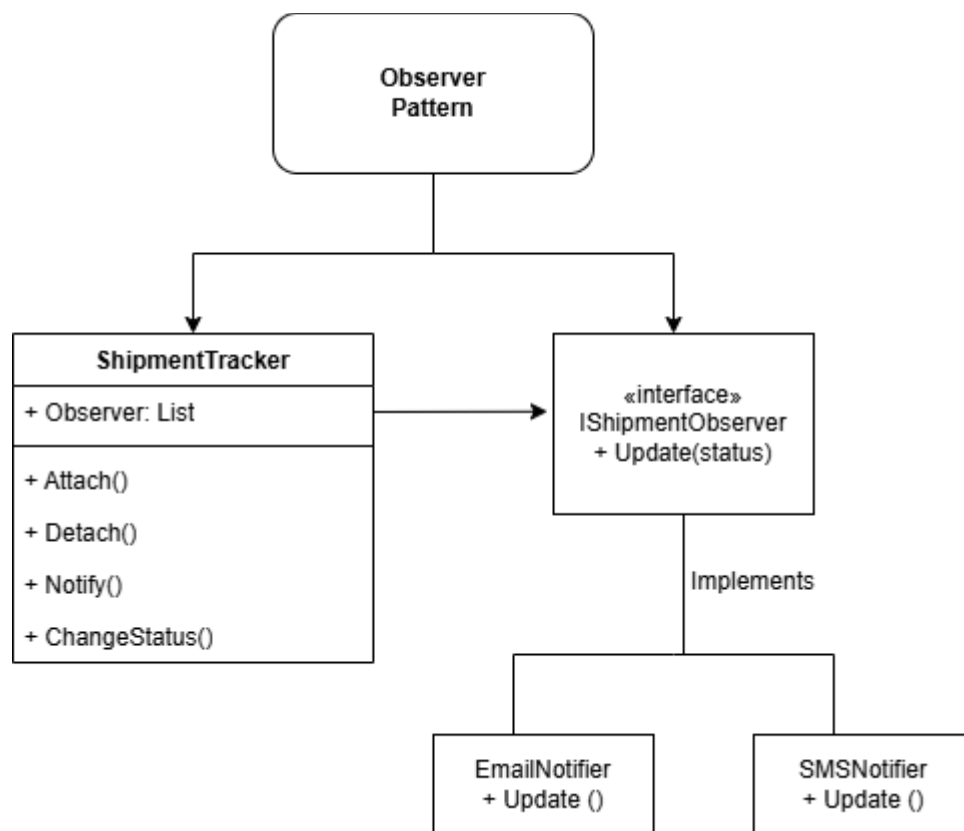
2: DESIGN PATTERN SELECTION

2.1 Pattern 1: Observer (behavioural)

Problem: When the status of a shipment changes, customers (email), drivers (SMS), and managers (dashboard) need notification. Hardcoding creates a tight coupling.

Solution: The entity (ShipmentTracker) maintains a list of observers (IShipmentObserver). Concrete observers (EmailNotifier, SMSNotifier) implement their own update logic. You can add new channels without changing existing code.

2.1.1 UML Class Diagram



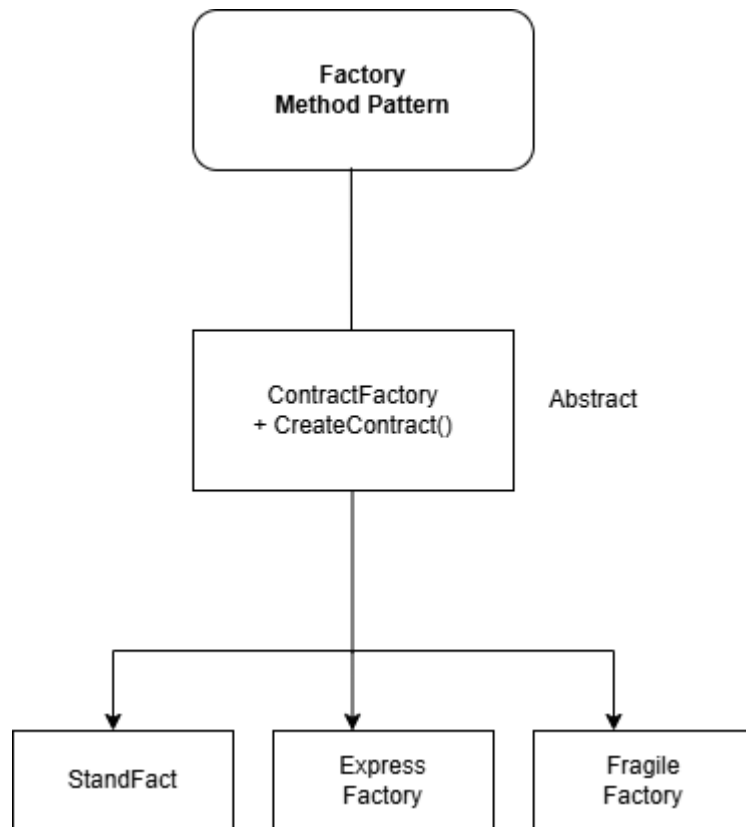
(Gamma, 1994)

2.2 Pattern 2: Factory Method (Creative)

Problem: Tech Move has three contract types (Standard, Express, Fragile-Goods), each with different validation rules and pricing. Using the new keyword disperses the creation logic.

Solution: An abstract ContractFactory with a CreateContract () method. Concrete factories (StandardContractFactory, ExpressContractFactory, FragileContractFactory) return the correct contract type. This centralizes creation and simplifies testing.

2.2.1 UML Class Diagram



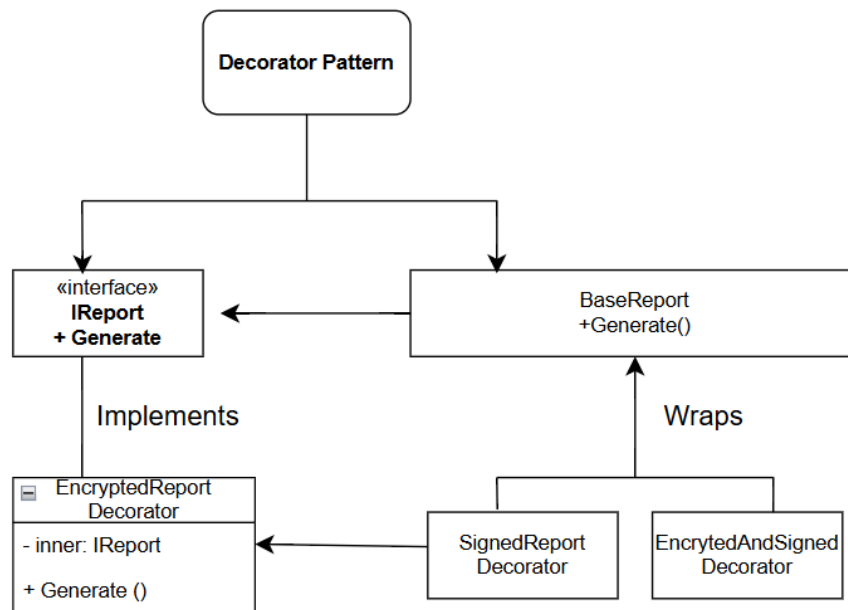
(Metsker, 2004)

2.3 Pattern 3: Decorator (structural)

Problem: Reports (operational, financial, compliance) may require encryption, digital signature, or both. Creating subclasses for every combination result in class explosion.

Solution: A BaseReport that implements IReport. Decorators (EncryptedReportDecorator, SignedReportDecorator) wrap the base object and add behavior at runtime. Example: new SignedReportDecorator (new EncryptedReportDecorator (new BaseReport ())) provides both functions.

2.3.1 UML Class Diagram



(Freeman, 2014)

3: NON-FUNCTIONAL REQUIREMENTS & SCALABILITY

3.1 Availability Requirements

Uptime measures system availability. Organization platforms must operate 24 hours a day.

Requirement	Target	Meaning
System Uptime	99.8%	Max 8.7 hours downtime per year
Recovery Time Objective	15 minutes	System recovers within 15 minutes of failure
Recovery Point Objective	5 minutes	Max 5 minutes of data loss in a disaster
Planned Maintenance	Zero downtime	Updates without stopping service

How to achieve your goals: Multiple Azure Availability Zones with active/active configuration, automated health checks that trigger failover in seconds, Azure SQL geo-replication, and blue-green deployments. (Azure, 2024), (Reintech, 2024), (MoldStud, 2025)

3.2 Interoperability Requirements

Interoperability means exchanging data with external systems (ERP, fleet tracking, customer portals, customs)

Requirement	Standard	Purpose
API Protocol	REST/HTTPS, GraphQL	Client and mobile integration
Data Format	JSON, XML	Data exchange
Authentication	OAuth 2.0, OpenID Connect	Secure third-party access
Fleet Tracking	MQTT	Real time vehicle telemetry
Customs Systems	EDI	Border clearance

API Gateway (Azure API Management) serves as a single point of entry, providing rate limiting, authentication, versioning, and protocol translation. (Azure, What is Azure API Management?, 2024), (Alumio, 2025), (Alumio, Composable Edifact Integrations, 2025), (Banks, 2017)

3.3 SCALABILITY: HANDLING 50,000 NEW CONTRACTS OVERNIGHT

If Tech Move acquires a competitor, GLMS is expected to record 50,000 contracts overnight.

The following five strategies work together:

Strategy 1 – Horizontal scaling: Kubernetes monitors CPU usage. New containers are automatically started when utilization exceeds 70% (for example, 10 to 50 containers in a few minutes).

Strategy 2: Load balancing. Azure load balancer distributes requests evenly across all containers and prevents individual containers from becoming overloaded.

Strategy 3 – Distributed cache. Redis caching saves frequently accessed data in memory, reducing the load on your database.

Strategy 4. Asynchronous processing: The received contract goes into an Azure Service Bus queue. The API returns an "authorization" response immediately. Background workers process contracts at a constant rate. The cue acts as a shock absorber.

Strategy 5 – Read database replicas: Report requests are routed to the read replicate database, leaving the primary database free for critical write operations.

Step by step process:

1. Contract Import API accepts the package and returns "accepted".
2. Each contract becomes a message in a queue.
3. Kubernetes detects CPU increase and starts more containers.
4. A load balancer distributes processing across all containers.
5. Employee checks the contract and writes it to the main database.
6. Report queries use read replicas.
7. Completion notification sent after processing

Result: The system continues to respond during the spike. No component is a bottleneck. (Azure, Azure Kubernetes Service (AKS) documentation., 2024), (Azure, Azure Cache for Redis documentation, 2024)

CONCLUSION

This report outlines a comprehensive technology strategy for Tech Move GLMS in three key areas:

Areas	Key Decisions
Architecture Framework	TOGAF selected for its step-by-step ADM process
Design Patterns	Observer, Factory Method, and Decorator
Non-Functional Requirements	99.9% availability, REST/EDI interoperability, five-part scalability

The TOGAF framework provides a step-by-step roadmap from vision to technology roadmap. The three Gang of Four design models address behavioural, creative, and structural issues. The non-functional analysis confirms that the system is designed for high availability, broad compatibility, and elastic scalability.

Tech Move GLMS, as designed in this article, can support current operations and future growth by providing a resilient, interoperable, and scalable platform for board investments.

REFERENCES

- Alumio. (2025). *Composable Edifact Integrations*. Retrieved from Composable Edifact Integrations: <https://www.alumio.com/nl/platforms/edifact>
- Alumio. (2025). *EDI Broker Integrations*. Retrieved from EDI Broker Integrations: <https://www.alumio.com/platforms/edi-broker>
- Azure, M. (2024). *Azure Cache for Redis documentation*. Retrieved from Microsoft learn: <https://learn.microsoft.com/en-us/azure/azure-cache-for-redis/>
- Azure, M. (2024). *Azure Kubernetes Service (AKS) documentation*. Retrieved from Azure Kubernetes Service (AKS) documentation.: <https://learn.microsoft.com/en-us/azure/aks/>
- Azure, M. (2024, April 12). *Design for high availability*. Retrieved from Microsoft Learn.: <https://learn.microsoft.com/en-us/azure/architecture/framework/resiliency/design-high-availability>
- Azure, M. (2024, April 12). *What is Azure API Management?* Retrieved from Microsoft Learn: <https://learn.microsoft.com/en-us/azure/api-management/api-management-key-concepts>
- Banks, A. a. (2017). *OASIS Standard*. Retrieved from OASIS Standard.: <https://docs.oasis-open.org/mqtt/mqtt/v5.0/os/mqtt-v5.0-os.html>

- Freeman, E. a. (2014). *Head First Design Patterns: A Brain-Friendly Guide. 2nd ed. Sebastopol, CA: O'Reilly Media.*
- Gamma, E. H. (1994). *Design Patterns: Elements of Reusable Object-Oriented Software. MA: Addison-Wesley., 1001.*
- Haren, V. (2018). The Open Group. *TOGAF Standard, 9.2.*
- Metsker, S. (2004). *Design Patterns in C#. Boston: Addison-Wesley.*
- MoldStud. (2025, October 29). *Essential Tips for Remote Developers - Getting Started with Azure SQL Database Geo-Replication.* Retrieved from MoldStud: <https://moldstud.com/articles/p-essential-tips-for-remote-developers-getting-started-with-azure-sql-database-geo-replication>
- Reintech. (2024, April 12). *Implementing High Availability Solutions in Azure SQL Server.* Retrieved from Reintech Media: <https://reintech.io/blog/implementing-high-availability-azure-sql-server>
- Standard, T. (2018, 1 9). *The Open Group.* Retrieved from The Open Group: <https://publications.opengroup.org/c182>
- Zachman, J. (1987). 'A Framework for Information Systems Architecture',. In J. Zachman, *A Framework for Information Systems Architecture'*, (p. 26). N/A: Zachman, J.A.
- Zachman, J. A. (1987). 'A framework for information systems architecture,,'. In J. A. Zachman, *IBM Systems Journal*, (pp. 276–292,).
- Zachman, T. v. (2024, 12 04). *Visual Paradigm.* Retrieved from A Comparative Analysis. Visual Paradigm Guides: <https://guides.visualparadigm.com>